Visum — Deep Interview Prep

Frontend Intern @ OpenFX — System Design, Data Flow & JD-Aligned Rehearsal

---

Every architecture claim, code snippet, and gap in this document is verified against the actual D:\visum repository — nothing here is invented. Sections are mapped directly to line items in the OpenFX Frontend Intern job description, so you can trace every talking point back to a requirement they'll actually be screening for.

*Boxes marked **HONEST GAP** flag places where the project doesn't yet do something the JD cares about. Say these out loud if asked — naming a gap with a concrete plan reads as far stronger than pretending it doesn't exist.*

Contents

1. Reading the JD Through Visum

OpenFX moves money across corridors: their frontend renders **rates, balances, and transaction states that update while the user is watching**, and a wrong number is a trust failure, not a cosmetic bug. Visum is a different domain — it renders an AI-visibility score instead of an FX rate — but the underlying frontend problem is the same shape: a number the user trusts, computed by a backend, that must be shown correctly, must handle every state between "nothing yet" and "here's your result," and must fail loudly and honestly rather than silently showing something wrong.

This table is the actual translation to have ready — it's the fastest way to make an unrelated project feel directly relevant to a fintech interviewer:

FX rate / balance	AI-visibility score (0–100) and its 8 underlying checks
Transaction state (pending/settled/failed)	Scan state (loading / redirect / error / loaded) — see Section 5
Numbers must never be wrong	lib/verification.ts refuses to render a metric unless it's actually present and valid
Live-updating data	Backend already tracks scan status (scan_tracker); frontend doesn't yet poll it — honest gap, Section 7
Money movement infra you plug into	A FastAPI scanning service with rate limiting, concurrency caps, and SSRF-safe URL validation

---

How to use this framing live

Don't force the metaphor into every answer — use it once, early, to signal you understand *why* this JD cares about the things it cares about, then let the rest of your answers stand on their own technical merit.

Q: This role is about financial data — how is a visibility scanner relevant?

> "The domain is different, but the frontend problem is the same: a number computed server-side that the user needs to trust, rendered through every state from 'nothing yet' to 'here's your result,' with a validation layer that refuses to show a value it can't verify. That's the same discipline FX rates or balances need — I just built it for a different kind of number."

2. What Visum Is

One breath: Visum scans any public URL and tells you, in about 20 seconds, whether AI systems like ChatGPT, Perplexity, and Claude can actually read your site — an "AI-visibility X-ray." It runs 8 technical checks (robots.txt AI-bot permissions, JSON-LD structured data, an llms.txt file, an MCP endpoint, whether content needs JavaScript to render, meta/Open Graph tags, sitemap.xml, and page load speed), turns the results into a 0–100 score with a letter band, and prioritizes fixes by impact-per-minute-of-effort.

Who it's for: non-technical store owners, founders, and marketing agencies who've noticed AI answer engines never mention their site.

This is deliberately a well-scoped portfolio project: one clear user, one measurable output, and a natural extension path — which is exactly why several dashboard-style pages exist as designed-ahead UI prototypes before the backend behind them exists (Section 6 explains exactly which ones and why).

3. System Design — High-Level Architecture

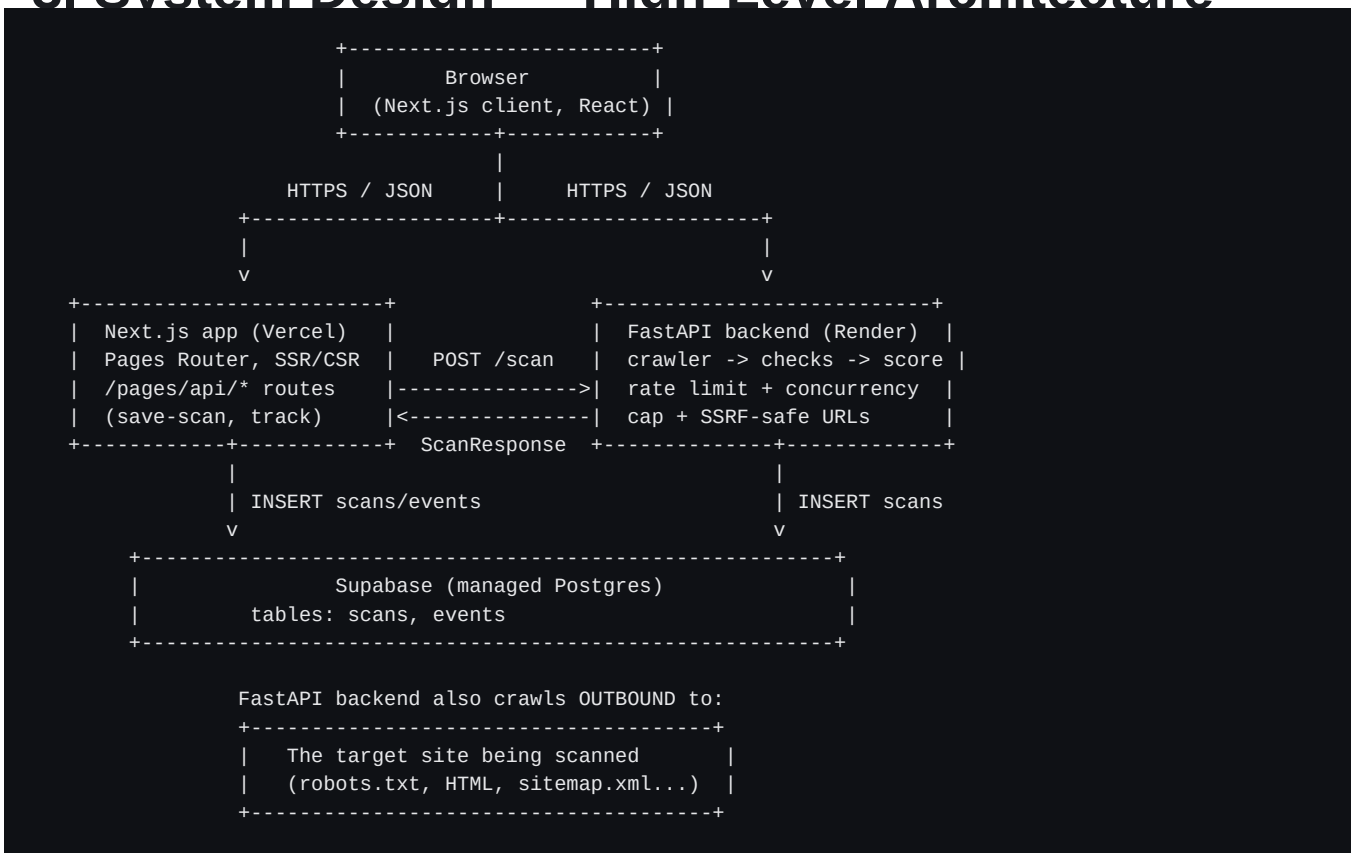

Fig. 1 — Visum's deployment topology: Vercel (frontend) + Render (backend) + Supabase (data), with the backend crawling out to whatever site the user submits.

3.1 Component responsibilities

Browser / React	Owns UI state (loading, error, form input), calls the backend, renders results, never trusts a partial payload (verification.ts gate).
Next.js server (API routes)	Two thin routes only — pages/api/save-scan.js and pages/api/track.js — acting as a backend-for-frontend so the browser never talks to Supabase directly with a privileged key.
FastAPI backend	The actual scanning engine: validates the URL (blocks localhost/private IPs), crawls it, runs 8 checks concurrently-safe, scores, validates its own output, optionally persists, returns JSON.
Supabase / Postgres	Durable storage for captured emails+scans (scans table) and analytics events (events table). Not used for auth.
Target site	An arbitrary third-party URL the backend fetches on the user's behalf — this is why SSRF protection (Section 4) is a real, non-optional concern.

3.2 Why this shape, not another

- **Thin API routes instead of direct client-to-Supabase calls:** keeps the privileged Supabase key server-side. (Worth noting honestly: the checklist review found `SUPABASE_SERVICE_KEY` currently sits in frontend/.env.local — a real, named security gap, good self-critique material.)
- **Separate FastAPI service instead of Next.js API routes doing the crawling:** crawling arbitrary third-party sites needs Playwright/headless-browser capability, real concurrency control, and a timeout budget — a long-running Python service on Render is a better fit than a serverless Vercel function with execution-time limits.
- **In-memory rate limiting and semaphores, not Redis:** single-instance deployment on Render at current scale; the code comments this explicitly ("Not distributed — resets on server restart. Sufficient for beta launch.") — a deliberate, stated trade-off rather than an oversight.

4. Data Flow — Every Path Traced End to End

4.1 The core scan flow

```
User types URL, clicks Scan
  |
  v
hero.tsx handleSubmit()
  1. validate URL client-side (new URL(...))
  2. setLoading(true); track('scan_initiated')
  3. await scanUrl(parsed.href)   <-- lib/api.js
        |
        v
     fetch POST /scan  { url }
        |
        v  (FastAPI, backend/app/main.py)
     1. rate limit check (IPRateLimiter, 30/hr/IP)
     2. _validate_url()  <- blocks localhost / private IPs (SSRF guard)
     3. acquire scan_semaphore (max 5 concurrent)
     4. asyncio.wait_for( crawl -> run_scan() , timeout=45s )
     5. optional INSERT INTO scans (Postgres)
     6. release semaphore, return ScanResponse JSON
        |
        v
  4. on success: sessionStorage.setItem('visum_result', JSON)
  5. on failure: setError(message); track('scan_error')
  6. finally: setLoading(false)
  7. onScanEnd(data) -> router navigates to /result
        |
        v
result.js reads sessionStorage, validates shape,
sets state = {type:'loaded'|'error'|'redirect'}, renders
```

Fig. 2 — The one fully real, end-to-end flow: index/hero -> POST /scan -> result.

4.2 The actual request/response shape

Verified shape (backend/app/schemas.py: ScanResponse / ScanRequest)

```
// Request:  POST https://visum-xoe3.onrender.com/scan
{ "url": "https://example.com" }

// Response: 200 OK, application/json
{
  "scan_id": "b3f1...",
  "url": "https://example.com",
  "total_score": 62,
  "band": "Good - Needs Work",
  "scan_time_ms": 4210,
  "checks": [
    { "name": "AI Bot Permissions (robots.txt)", "passed": true,  "partial": false, "points": 15 },
    { "name": "JSON-LD Structured Data",         "passed": false, "partial": true,  "points": 8  },
    ...
  ]
}
```

4.3 Email capture flow

```
EmailCapture form (in result.js)
  submit -> validate email regex client-side
  setSaving(true)
  fetch POST /api/save-scan  { scan_id, url, email, total_score, band, checks }
        |
        v  (Next.js API route, pages/api/save-scan.js)
     supabase.from('scans').upsert(...)
        |
        v
  200 -> setSaved(true), track('email_captured')
  !ok -> setError('Something went wrong...')
  finally -> setSaving(false)
```

4.4 Analytics flow

```
Any page: track(event, props)   (lib/analytics.js)
  fetch POST /api/track  { event, properties }  .catch(()=>{})   <- fire-and-forget
        |
        v  (pages/api/track.js)
     supabase.from('events').insert(...)
  Never blocks the UI and never surfaces a failure to the user —
  a deliberate choice: analytics must not be able to break the product.
```

4.5 Status-polling path — built on the backend, unused by the frontend

The backend already exposes `GET /status/{scan_id}`, backed by an in-memory `scan_tracker` dict that records `PENDING` → `RUNNING` → `COMPLETE/FAILED` for every scan. This is real, shipped backend infrastructure for exactly the kind of live, in-progress state the JD asks about — the frontend just doesn't call it yet; it does one synchronous `await scanUrl()` instead. This is covered in depth in Section 7 as a named, honest gap with a concrete proposed implementation.

5. Frontend Deep Dive — Components, Hooks, Re-renders

5.1 The discriminated-union state pattern (result.js)

Instead of separate booleans (`loading`, `error`, `data`) that could contradict each other, the results page models its state as one object with a `type` tag — a discriminated union. Only one state is ever true at once, so there's no "loading is true AND error is set" bug class.

frontend/pages/result.js

```
const [state, setState] = useState({ type: "loading" });

useEffect(() => {
  const id = setTimeout(() => {
    try {
      const stored = sessionStorage.getItem("visum_result");
      if (!stored) { setState({ type: "redirect" }); return; }
      const parsed = JSON.parse(stored);
      if (!parsed.result || typeof parsed.result.total_score !== "number") {
        setState({ type: "error", message: "Invalid scan data received. Please scan again." });
        return;
      }
      setState({ type: "loaded", data: parsed });
    } catch {
      setState({ type: "error", message: "Failed to load scan results. Please scan again." });
    }
  }, 0);
  return () => clearTimeout(id);
}, []);

if (state.type === "redirect" || state.type === "loading") return null;
if (state.type === "error") return <ErrorState message={state.message} />;
const { result } = state.data;   // TypeScript would narrow this automatically
```

Notice the shape validation before trusting the payload — `typeof parsed.result.total_score !== "number"` — that's the same "don't render what you can't verify" instinct as `lib/verification.ts`, applied inline instead of through the shared guard module. Worth naming that inconsistency if asked "how would you improve this": consolidate both into the same verification layer.

5.2 Context: sidebar-context.tsx — the one real Context in the app

frontend/components/sidebar/sidebar-context.tsx

```
const SidebarContext = createContext<SidebarContextValue | null>(null);

export function SidebarProvider({ children }: { children: ReactNode }) {
  const [expanded, setExpanded] = useState(true);
  const [isMobile, setIsMobile] = useState(false);

  useEffect(() => {
    const check = () => setIsMobile(window.innerWidth < 768);
    check();
    window.addEventListener("resize", check);
    return () => window.removeEventListener("resize", check);
  }, []);
```

```
  useEffect(() => {
    function handleKeyDown(e) {
      if ((e.metaKey || e.ctrlKey) && e.key === "b") {
        e.preventDefault();
        setExpanded((prev) => !prev);
      }
    }
    window.addEventListener("keydown", handleKeyDown);
    return () => window.removeEventListener("keydown", handleKeyDown);
  }, []);
  // ...
}

export function useSidebar() {
  const ctx = useContext(SidebarContext);
  if (!ctx) throw new Error("useSidebar must be used within SidebarProvider");
  return ctx;
}
```

- Every effect with an empty dependency array (`[]`) runs exactly once on mount and cleans up on unmount — both the resize listener and the keydown listener follow this, which is why they safely add/remove without leaking across re-renders.
- `useSidebar()` throwing when called outside a provider is a real, deliberate "fail loud in development" pattern — worth calling out if asked about defensive coding habits.

6. Edge States: Loading / Empty / Error / Stale

JD FOCUS: "Cover loading, empty, error, and stale states, which is where most frontend bugs in financial products live."

This is arguably the single most important line in the JD to have a strong, concrete answer for. Here's exactly what Visum does for three of the four states, and an honest gap for the fourth.

Loading

Two different loading patterns exist, and it's worth being able to name both:

- **Real, indeterminate loading** — `hero.tsx`'s `loading` boolean, true for the actual duration of the network call to `/scan` (a few seconds, backend-dependent). This is a true async loading state.
- **Simulated loading** — the mocked dashboard pages (`dashboard.js`, etc.) use `setTimeout(() => setLoading(false), 1800)` to show a skeleton before rendering static data. Be upfront that this is simulated, not a real fetch — it exists to design the loading UI ahead of a backend that doesn't exist yet for those pages.

Error

Two distinct error surfaces, at two different granularities:

Two granularities of error UI, both real

```
// Page-level error — result.js
function ErrorState({ message }) {
  return (
    <div className="min-h-screen flex items-center justify-center ...">
      <h2>Something went wrong</h2>
      <p>{message}</p>
    </div>
  );
}

// Field-level error — EmailCapture, inline next to the input
{error && <p className="text-destructive" role="alert">{error}</p>}
```

- `role="alert"` on the inline error is a real accessibility detail — screen readers announce it immediately when it appears.
- `scanUrl()` in `lib/api.js` throws on any non-OK response with the backend's actual error detail, so the UI shows the real failure reason (rate-limited, invalid URL, crawler blocked) rather than a generic message.
- In `hero.tsx`, note the comment: *"Navigate outside try/catch — a navigation error must not mask a successful scan."* That's a specific, considered edge case: don't let a router.push() failure look like the scan itself failed.

Empty

> **HONEST GAP — No dedicated empty state**
>
> There's no scenario in the current flow where a scan legitimately succeeds but returns zero checks or zero data — every successful response always has all 8 checks. The honest answer, if asked, is that the closest thing to an empty state is the redirect case in result.js (no sessionStorage entry — user hasn't scanned anything yet), which sends them back to the homepage rather than showing an empty results page. A good follow-up: "if I added scan history, I'd need a real empty state — 'You haven't scanned anything yet' with a CTA — for a first-time user on that page."

Stale

HONEST GAP — No staleness indicator on results

A scan result, once stored in sessionStorage, is shown as current with no timestamp or 'this may be out of date' signal, and there's no re-scan-on-return behavior. For an FX product this would be a real bug (a stale rate is a wrong rate); for Visum it's lower-stakes since a score doesn't change minute to minute, but it's still a legitimate gap. The concrete fix: store a timestamp alongside the scan result and show a banner past some threshold —

Proposed, not shipped — label it exactly that way if you show this

```
const ageMs = Date.now() - scan.scannedAt;
const isStale = ageMs > 1000 * 60 * 60 * 24; // 24h

{isStale && (
  <div role="status" className="stale-banner">
    This result is from {formatRelative(scan.scannedAt)}.
    <button onClick={handleScanAgain}>Rescan now</button>
  </div>
)}
```

7. Live & Real-Time Data

JD FOCUS: "Render rates, balances, and transaction states that update while the user is watching." / "Any experience with real-time data (WebSockets, polling, SSE)"

Today's flow is a single synchronous request/response: the browser awaits one `POST /scan` call for the whole 4–second-ish scan duration, with a loading spinner covering that entire window. That's honest and fine for a scan measured in single digit seconds — but it's not "live data updating while the user watches," it's "wait, then show the final answer."

What already exists to build real-time on top of

The backend's `scan_tracker` dict and `GET /status/{scan_id}` endpoint (Section 4.5) already model exactly the state machine a live progress UI needs: `PENDING` → `RUNNING` → `COMPLETE / FAILED`. The missing piece is entirely on the frontend.

Proposed implementation — polling (simplest fit for this backend)

Proposed — the cancelled-flag cleanup matters: without it, a component that unmounts mid-poll would call setState on an unmounted component.

```
async function pollScanStatus(scanId, { intervalMs = 1500, timeoutMs = 60000 } = {}) {
  const start = Date.now();
  while (Date.now() - start < timeoutMs) {
    const res = await fetch(`${API_URL}/status/${scanId}`);
    const status = await res.json();
    if (status.status === "complete" || status.status === "failed") return status;
    await new Promise((r) => setTimeout(r, intervalMs));
  }
  throw new Error("Scan timed out client-side");
}

// In the component:
useEffect(() => {
  if (!scanId) return;
  let cancelled = false;
  pollScanStatus(scanId).then((status) => {
    if (!cancelled) setScanStatus(status);
  });
  return () => { cancelled = true; };   // avoid setState after unmount
}, [scanId]);
```

Why polling over WebSockets/SSE here, and when that answer flips

- A scan takes single-digit seconds and happens once per user action — polling every 1–2s for that duration is cheap and simple, with no persistent-connection infrastructure to run.
- WebSockets or SSE earn their cost when updates are frequent, ongoing, and server-initiated over a long-lived session — exactly the OpenFX case: a live FX rate ticking every second, or a transaction status that can change at any time while the page is just sitting open. If asked directly, say this distinction plainly: "I'd reach for SSE for a continuously updating value like a live rate, and reserve polling for a bounded, one-shot operation like a scan."
- SSE over WebSockets specifically for one-directional server-to-client updates (like a rate feed) is usually the simpler choice — plain HTTP, auto-reconnect built into the `EventSource` API, no need for the bidirectional complexity WebSockets add when the client never needs to push anything back.

8. Testing & CI

JD FOCUS: "Write test cases... covering the normal path as well as failure and edge cases" / "Automate those test cases so they run in CI"

This is the JD's heaviest emphasis, and it's also where the honest gap in this project is largest — said plainly, up front, is the strongest way to handle it.

8.1 What's real: backend testing

`backend/tests/` has six files — `test_api.py, test_checks.py, test_crawler.py, test_scorer.py, test_validation.py`, and a property-based test file (`test_property.py`, almost certainly Hypothesis-style generated-input testing) — run via pytest, and wired into actual CI:

.github/workflows/ci.yml — real, runs on every push/PR to main

```
# .github/workflows/ci.yml
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  test:
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.14" }
      - run: pip install -r backend/requirements.txt
      - run: cd backend && python -m pytest tests/ -v --tb=short
```

8.2 The frontend gap

> **HONEST GAP — Frontend tests exist but aren't wired to a runner or CI**
>
> frontend/__tests__/verification.test.ts is a real file testing the canRenderX() guards in lib/verification.ts, but frontend/package.json has no test script and no Jest/Vitest devDependency — and the CI workflow above only runs backend pytest. So this test file cannot currently be executed by anyone, including CI. This is the single most direct, defensible thing to bring up yourself: "the frontend has a test file, but it's disconnected from a runner and from CI — that's exactly the kind of gap this role is meant to close, and it's the first thing I'd fix."

What you'd add (say this as a plan, not as done)

vitest was chosen over Jest here because it shares Vite's config/transform pipeline and is the natural fit alongside Astro tooling (Astro's own test docs recommend it) — relevant to bring up given the JD's Astro focus.

```
// package.json additions
"scripts": { "test": "vitest run", "test:watch": "vitest" },
"devDependencies": {
  "vitest": "^2.0.0",
  "@testing-library/react": "^16.0.0",
  "@testing-library/jest-dom": "^6.0.0",
  "jsdom": "^25.0.0"
}
```

Proposed test file, not shipped — shows the normal path and one failure path per the JD's ask

```
// Example: normal path + failure path for the real scan flow
import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import Hero from "../components/hero";
import { scanUrl } from "../lib/api";

vi.mock("../lib/api");

test("shows a loading state, then success, on a valid scan", async () => {
  scanUrl.mockResolvedValue({ result: { total_score: 80, band: "Good" } });
  render(<Hero />);
  fireEvent.change(screen.getByPlaceholderText(/domain/i), { target: { value: "example.com" } });
  fireEvent.click(screen.getByRole("button", { name: /scan/i }));
  expect(screen.getByRole("button")).toBeDisabled(); // loading
  await waitFor(() => expect(sessionStorage.getItem("visum_result")).toBeTruthy());
});

test("shows the backend\'s real error message on failure", async () => {
  scanUrl.mockRejectedValue(new Error("Server busy. Please retry later."));
  render(<Hero />);
  fireEvent.change(screen.getByPlaceholderText(/domain/i), { target: { value: "example.com" } });
  fireEvent.click(screen.getByRole("button", { name: /scan/i }));
  expect(await screen.findByText(/server busy/i)).toBeInTheDocument();
});
```

If pushed further: "I'd also add this as a required check in the same GitHub Actions workflow, as a second job alongside the backend pytest job, so a PR can't merge with a broken frontend test."

9. JavaScript Fundamentals in This Codebase

JD FOCUS: "Solid foundation in JavaScript: async behaviour, array methods, objects"

Async behaviour — async/await with proper error boundaries

lib/api.js — note: fetch() only rejects on network failure, never on 4xx/5xx, so response.ok must be checked explicitly and the error re-thrown for the caller's try/catch

```
export async function scanUrl(url) {
  const response = await fetch(`${API_URL}/scan`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ url }),
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.detail || "Scan failed");
  }
  return response.json();
}
```

If asked "what's a common mistake with fetch and error handling," this file is the answer: forgetting that `fetch` resolves normally even for a 500 response, so without the explicit `if (!response.ok) throw ...`, a failed scan would silently be treated as a success.

Array methods — filter, used for both categorization and counting

frontend/pages/result.js

```
function categorizeChecks(checks) {
  return {
    performance: checks.filter((c) => PERFORMANCE_CHECKS.includes(c.name)),
    seo: checks.filter((c) => SEO_CHECKS.includes(c.name)),
    aiVisibility: checks.filter((c) => AI_VISIBILITY_CHECKS.includes(c.name)),
  };
}

const totalFailed = result.checks.filter((c) => !c.passed && !c.partial).length;
const totalWarnings = result.checks.filter((c) => c.partial).length;
```

Worth noting if pushed on Big-O: this re-filters the full checks array three separate times for categorization, plus twice more for counts — five passes over a small (8-item) array, which is fine at this size but would be worth collapsing into a single `reduce()` if the list ever grew large or ran per-render on something hot.

Objects — discriminated unions, destructuring, spread

Patterns pulled from result.js and the ScoreEstimator island (Section 11)

```
const { result } = state.data;                       // destructuring
setChecked((prev) => ({ ...prev, [s.key]: e.target.checked }));  // spread + computed key
const checkCounts = { total, passed, failed: totalFailed, warnings: totalWarnings };
```

10. React Fundamentals: Why a Re-render Happens

JD FOCUS: "Working knowledge of React: components, props, state, hooks, and why a re-render happens"

The three triggers, each with a real example from this codebase:

1. State changes (useState)

EmailCapture, frontend/pages/result.js

```
const [saving, setSaving] = useState(false);
setSaving(true);   // schedules a re-render of EmailCapture with saving=true
// the submit button reads {saving} to switch between a spinner and the label "Save"
```

Calling a state setter with a value that's `Object.is`-equal to the current value does **not** trigger a re-render — React bails out. This matters for the sidebar's `toggleGroup`, which builds a brand-new `Set` on every toggle specifically so the reference changes and the re-render actually fires:

sidebar-context.tsx — mutating `prev` in place and returning it would NOT re-render, since the reference wouldn't change

```
const toggleGroup = useCallback((id) => {
  setExpandedGroups((prev) => {
    const next = new Set(prev);       // new reference, even if same members
    if (next.has(id)) next.delete(id); else next.add(id);
    return next;
  });
}, []);
```

2. Props changes (parent re-renders, passes new props)

When `SidebarProvider`'s `expanded` state changes, every component reading it via `useSidebar()` re-renders — props/context propagate downward, and React re-renders a component whenever the value it's subscribed to changes, regardless of whether that specific component's own state changed.

3. Context changes (useContext)

Every consumer of `SidebarContext` re-renders whenever the **context value object** changes — which includes if the provider re-creates that object on every render even when nothing meaningful changed. This is a classic Context perf trap; the provider here rebuilds the value object inline on every render:

sidebar-context.tsx — could be wrapped in useMemo() to avoid re-rendering every consumer whenever an unrelated piece of provider state changes

```
return (
  <SidebarContext.Provider
    value={{ expanded, setExpanded, toggle, mobileOpen, ... }}  // new object every render
  >
```

This is a genuinely good thing to point out proactively if asked to critique the codebase: "the context value is rebuilt on every render, so any consumer re-renders even for unrelated state changes in the provider — wrapping it in useMemo keyed on its dependencies would fix that."

Why a component does NOT re-render

- A child wrapped in `React.memo` skips re-rendering when its props are shallow-equal to the previous render, even if its parent re-rendered.
- A ref update (`useRef`, e.g. `intervalRef` in hero.tsx) never triggers a re-render by itself — that's precisely why refs are used for values like interval IDs that need to persist across renders without causing one.

11. Astro + React Islands — Built for This Interview

JD FOCUS: "Astro with React: familiarity with building pages in Astro and using React components as interactive islands" / Nice to have: "Experience shipping an Astro project, or working with SSR and partial hydration"

Visum's main app is Next.js, not Astro — so rather than stretch that, a small, real Astro project was built alongside it at `D:\visum\astro-docs`, specifically to have genuine, working, explainable Astro experience for this interview. Be precise about scope if asked: this is a purpose-built companion project, not a rewrite of Visum.

11.1 Islands architecture, precisely

Astro's default output is plain HTML with **zero client-side JavaScript**. A page is built once (at build time, or per-request in server mode) and shipped as static markup. A component only ships JavaScript and hydrates on the client if a page explicitly opts it in with a `client:*` directive — becoming an isolated "island" of interactivity inside an otherwise inert page.

client:load	Hydrate immediately on page load. Above-the-fold interactivity (nav toggle, search box).
client:visible	Hydrate once the component scrolls into view, via IntersectionObserver. Used for the ScoreEstimator below — no reason to pay for its JS before the user reaches it.
client:idle	Hydrate when the browser is idle (requestIdleCallback). Lower-priority interactivity.
client:only="react"	Skip server rendering entirely, client-render only. Needed when a component touches window/localStorage that would break during SSR.

This is the core value proposition over a plain React app: Next.js/CRA hydrate the whole tree by default; Astro flips that default to static, JS opt-in per component — which is why content-heavy Astro sites ship dramatically less client JavaScript.

11.2 What was actually built

```
astro-docs/
  astro.config.mjs                    - registers @astrojs/react
  src/layouts/BaseLayout.astro        - shared shell (header, nav, footer, global CSS)
  src/components/ScoreEstimator.tsx   - the one React island
  src/pages/index.astro               - landing page embedding the island
  src/pages/docs/getting-started.astro - a second, fully static page
```

```
// src/components/ScoreEstimator.tsx  — real, stateful React
const SIGNALS = [
  { key: 'llmsTxt', label: 'Has an llms.txt file', weight: 20 },
  { key: 'structuredData', label: 'Uses structured data (schema.org)', weight: 20 },
  { key: 'crawlable', label: 'Content is server-rendered / crawlable', weight: 25 },
  { key: 'citations', label: 'Cited by other sites or AI answers', weight: 20 },
  { key: 'freshness', label: 'Content updated in the last 90 days', weight: 15 },
];
```

```
export default function ScoreEstimator() {
  const [checked, setChecked] = useState({});
  const score = useMemo(
    () => SIGNALS.reduce((t, s) => t + (checked[s.key] ? s.weight : 0), 0),
    [checked]
  );
  const band = score >= 80 ? 'Strong' : score >= 50 ? 'Developing' : 'At risk';
  // renders checkboxes + live score + band
}
```

```
---
// src/pages/index.astro
import BaseLayout from '../layouts/BaseLayout.astro';
import ScoreEstimator from '../components/ScoreEstimator.tsx';
---
<BaseLayout title="AI Visibility Score Estimator">
  <h1>AI Visibility Score Estimator</h1>
  <p>Rendered entirely at build time — the checklist is the only interactive part.</p>

  <ScoreEstimator client:visible />

  <h2>Why Astro here</h2>
  <p>Everything above ships as plain HTML with zero client JS.</p>
</BaseLayout>
```

`client:visible` was the deliberate choice, not `client:load`: the estimator sits below an intro paragraph, so there's no reason to pay for its JS before a reader scrolls to it. State the reasoning, not just the directive, if asked why.

11.3 Verified working

Built with `npm run build` (compiled both routes to static HTML in `dist/`) and ran with `npm run preview`. Clicking the checkboxes updated the score live (0 → 45 after checking two boxes), confirming the island genuinely hydrates and holds client-side state — not just static markup.

11.4 SSR and partial hydration (the JD's "nice to have")

Astro supports two output modes, set in `astro.config.mjs`:

astro-docs/astro.config.mjs

```
export default defineConfig({
  output: "static",  // default: prerender everything at build time (used here)
  // output: "server", // or "hybrid": render per-request, e.g. for logged-in/dynamic content
  integrations: [react()],
});
```

"Partial hydration" is just another name for the islands model itself — the page is fully rendered (statically or per-request), and only the opted-in islands are hydrated with JS on the client; the rest of the DOM is never touched by React at all. If asked to contrast Astro's SSR with Next.js's: Next.js's App Router Server Components render on the server and ship zero JS by default too, but the mental model in Astro is page-level (whole `.astro` files are server-only by default, with islands as the explicit escape hatch), whereas Next.js Server/Client Components are decided per-file with a directive at the top.

11.5 How this maps to your React experience

The hands-on muscle memory — component composition, hooks, controlled inputs, conditional rendering — comes from the full Visum frontend. Astro islands mount the same React components through a more selective hydration boundary; the new skill isn't React itself, it's deciding *which* components deserve a hydration boundary and *when* they should activate — exactly the reasoning behind the `client:visible` choice above.

12. HTTP, JSON & Reading the Network Tab

JD FOCUS: "Basic understanding of HTTP and JSON, and able to read a network call in DevTools"

Be ready to actually open DevTools — Network tab — on the deployed Visum site, submit a scan, and narrate what you see. Here's what's really there:

- **Request:** `POST https://visum-xoe3.onrender.com/scan`, `Content-Type: application/json`, body `{"url": "..."}`.
- **Status codes actually used by the backend:** `200` success, `400` invalid URL, `408` scan timed out (45s global timeout), `429`-style behavior via the rate limiter (check the exact status the endpoint returns before claiming a specific code), `503` "Server busy" when the concurrency semaphore times out waiting for a slot.
- **Response headers:** CORS headers (`Access-Control-Allow-Origin`) since the frontend on Vercel and backend on Render are different origins — worth being able to explain what a CORS error looks like in the console and why `ALLOWED_ORIGINS` exists in the backend's env config.
- **Timing:** the scan genuinely takes a few seconds — visible as real latency in the Network tab's waterfall, not an instant mock response.

Q: Open DevTools and show me the network request for a scan.

Walk through: Network tab → filter Fetch/XHR → submit a scan → click the /scan request → Headers tab (method, status, request payload) → Response tab (the JSON shape from Section 4.2) → Timing tab (how long it actually took). Narrate each tab out loud as you click it.

13. Git Workflow

JD FOCUS: "Familiarity with Git: branch, commit, pull request" / "Follow the team's Git workflow"

The repo's own history is real evidence here — recent commits include a feature branch merged through a pull request (`Merge pull request #1 from UTSAV-2024/redesign/frontend-instrument`), plus a sequence of focused, single-purpose commits (a chore pinning a launch config port, a fix for a type error, a feature commit for a design-system rebuild). That's a real, demonstrable pattern: branch per feature, PR to merge, small focused commits rather than one giant commit.

Q: Walk me through your Git workflow on this project.

> "I branch per feature or redesign — for example the frontend redesign went through a branch called redesign/frontend-instrument, merged back to main through a pull request rather than a direct push. I keep commits scoped to one concern each — a chore commit for a config change is separate from a fix commit for a type error, so the history stays easy to bisect or revert."

14. Common System Design Questions — Answered From This Project

Q: How would you scale the scan service if traffic grew 100x?

"First, the in-memory rate limiter and concurrency semaphore would need to move to something shared — Redis — since right now they reset per-instance and don't coordinate across multiple backend instances. Second, I'd run the FastAPI service behind a load balancer with several instances, since the crawl-and-score work is CPU/network-bound per request and horizontally scales well. Third, I'd cache scan results for a URL for some TTL — rescanning the same URL from different users within an hour doesn't need a fresh crawl every time."

Q: How would you add caching to avoid re-scanning the same URL repeatedly?

"Key a cache (Redis, or even the existing Postgres scans table) by a normalized URL, store the result with a timestamp, and on a new scan request, check the cache first — if a result exists and is younger than some TTL (say, 1 hour), return it immediately instead of re-crawling. I'd still let the user force a fresh scan explicitly, which ties back to the 'stale' state gap from Section 6 — the UI would need to show 'cached, N minutes old' versus 'fresh.'"

Q: How would you design the database schema for scan history?

"There's already a scans table (scan_id, url, total_score, band, checks as JSON, scan_time_ms, created_at). For real history I'd add a user_id or email foreign key and index on (email, created_at) for a 'my scan history' query, plus an index on url for the caching use case above. The checks column being JSON rather than a normalized child table is a reasonable trade-off here — the check list has a fixed, small shape, so normalizing it would add join complexity for little benefit."

Q: How do you prevent this scanning service from being abused as an SSRF vector?

"That's already handled — _validate_url() in main.py parses the submitted URL and rejects localhost, loopback, and private IP ranges before the crawler ever touches it. Without that, someone could submit 'http://169.254.169.254/' or 'http://localhost:5432' and use the scanner as a proxy to probe internal infrastructure or cloud metadata endpoints."

Q: How would you handle two users scanning the same URL at the same time?

"Right now they'd just run as two independent concurrent scans, each consuming a slot in the 5-slot semaphore — correct but wasteful. A better design would be to de-duplicate in-flight requests: if a scan for that URL is already RUNNING in scan_tracker, have the second request attach to the same in-progress result instead of starting a second crawl, and resolve both callers when it completes."

Q: What would you change about the frontend/backend boundary?

"Right now email capture and analytics go through Next.js API routes to Supabase, while the scan itself goes directly to FastAPI — two separate backend surfaces. I'd consider whether save-scan and track should actually live on the FastAPI service instead, so there's one backend of record instead of two, but the current split does correctly keep the privileged Supabase key out of the browser either way."

Q: How would you design a live-updating balance/rate ticker, generalizing from your polling proposal?

"For a continuously updating value I'd move off my earlier polling proposal to Server-Sent Events — the server pushes each new rate as it changes over one long-lived HTTP connection, the client just does new EventSource(url) and updates state in the onmessage handler. I'd debounce or throttle the resulting re-renders if updates arrive faster than the UI needs to reflect them, and I'd show a 'reconnecting...' state if the connection drops, since EventSource auto-reconnects but there's a gap while it does."

15. Full JD-Aligned Rehearsal

Q: Walk me through a project you're proud of, line by line if needed.

"Visum is an AI-visibility scanner — give it a URL, it runs 8 checks and returns a 0–100 score with prioritized fixes. The core flow: hero.tsx validates the URL, calls scanUrl() in lib/api.js, which POSTs to a FastAPI backend that validates the URL against SSRF, rate-limits by IP, crawls the site, runs the checks, scores them, and returns JSON. The frontend stores that in sessionStorage and the result page reads it through a discriminated-union state machine — loading, redirect, error, or loaded — gated by a verification layer that refuses to render any field it can't confirm is real. I can open any of those files and explain every line."

Q: Have you used Astro? Have you built pages with React islands?

"Yes — I built a small Astro site alongside my main Next.js project specifically to work with islands architecture: a static docs page with zero client JS, and a landing page with one React component, a live scoring checklist, mounted as an island with client:visible so its JS only loads once it scrolls into view. I can walk through exactly how that's wired and why I chose that directive."

Q: Tell me about a time you had to debug something you didn't write.

"When I reviewed my own dashboard pages, I found several places where a check's explanation copy was pulled from a shared HUMAN_EXPLANATIONS lookup table but repeated the same generic sentence for both a partial and a full failure of the same check — it read as a real bug that would confuse a user, even though the code 'worked.' I logged it in an improvement checklist rather than silently patching it, since fixing the copy properly needed per-status text, not a quick hack."

Q: How do you approach code review — giving and receiving?

"On the receiving end, my recent commit history shows small, single-purpose commits — e.g. a type fix isolated from a feature commit — specifically so a reviewer can evaluate one change at a time. I'd want feedback that's specific to a line or a behavior, and I try to write PR descriptions that make the 'why' obvious so a reviewer doesn't have to reverse-engineer intent from the diff."

Q: What could break in the scan feature that isn't obviously broken today?

"A few things: the in-memory rate limiter and scan_tracker reset on every server restart or redeploy, so a user's rate-limit window silently resets — not dangerous, but not durable either. The frontend's shape check on the stored scan result (typeof total_score === 'number') would pass a malformed-but-numeric score through without validating its range, whereas lib/verification.ts's canRenderMetric does support min/max bounds — that inline check in result.js should really route through the same shared guard."

Q: Why should we trust you with production code in a financial product after 3–6 months?

"Because I already build with the instinct this JD is describing — a validation layer that refuses to render unverified data, a discriminated-union state machine instead of ad hoc booleans, explicit handling of the 'what if the network call itself fails to navigate after success' edge case. Those aren't things I did because a job description told me to; they were already how I built this before I read your JD. What I'd be learning here is the FX domain and your team's specific review process, not the fundamental discipline of not lying to the user with the UI."

Closing Note

Every technical claim in this document maps to a real file in D:\visum or D:\visum\astro-docs, and every HONEST GAP box names something true rather than papering over it. If a question goes somewhere this document doesn't cover, the same rule applies live: state what you actually know, reason about the rest out loud, and never extend a real fact into an invented one. That instinct — demonstrated, not just claimed — is exactly what a 3-to-6-month hiring pipeline for a trust-critical product is screening for.